Universidad Modelo
Escuela de Ingeniería · Ingeniería en Desarrollo de Tecnología y Software

CLUSTERING

Proyecto #2 · Aprendizaje no supervisado con K-means

Asignatura: Inteligencia artificial · Profesor: Edson G. Estrada López

Martínez, Esteban · Osorio, Javier · Sosa, César

Fecha de elaboración: 26 de octubre de 2026

qué buscamos

Objetivos

Objetivo general

Aplicar el algoritmo de agrupamiento K-means en la solución de problemas de aprendizaje no supervisado.

Objetivos particulares

- Comprender los fundamentos del aprendizaje no supervisado y del algoritmo K-means, e implementarlo desde cero en Python, incluyendo dos métodos para elegir el número óptimo de clústeres (codo y silueta).
- Aplicar K-means mediante una función de librería (scikit-learn) a dos problemas de elección libre, interpretando e evaluando los grupos obtenidos.

Descripción y requerimientos

El proyecto investiga el aprendizaje no supervisado y profundiza en K-means: se implementa el algoritmo y los criterios de selección de K sin librerías, y se aplica con scikit-learn a la reducción de colores de una imagen y al descubrimiento de roles de combate en 1,025 Pokémon.

#	Requerimiento solicitado	Implementado	Evidencia
1	Investigar aprendizaje no supervisado (generalidades, aplicaciones, algoritmos)	Sí	Diapositivas 4–6
2	Investigar K-means (conceptos, fundamentos matemáticos, aplicaciones, ventajas y desventajas)	Sí	Diapositivas 7–9
3	Investigar dos algoritmos para elegir K e implementarlos en Python sin librería	Sí: codo y silueta	Diapositivas 10–11
4	Implementar K-means sin librería y visualizar su operación en una nube de puntos	Sí	Diapositiva 12
5	Aplicar K-means (función de librería) a dos problemas de elección libre	Sí: colores y Pokémon	Diapositivas 13–17

Aprendizaje no supervisado

Analiza datos sin etiquetar y descubre por sí solo patrones, grupos y relaciones, sin intervención humana.

Sin etiquetas

Los datos llegan sin respuestas predefinidas: solo se dispone de las características de cada observación.

Estructura propia

El modelo infiere reglas a partir de similitudes y diferencias entre las observaciones.

Sin salida conocida

A diferencia del aprendizaje supervisado, no hay un objetivo fijado de antemano; se evalúa con métricas internas.

dónde se usa

Aplicaciones

Segmentación de clientes

Perfiles de comprador según su comportamiento de compra.

Detección de anomalías y fraude

Datos atípicos que revelan fallas, errores o transacciones sospechosas.

Motores de recomendación

Reglas de asociación para venta cruzada y sugerencias.

Lenguaje natural

Agrupar artículos en secciones de noticias, como Google News.

Imágenes y visión

Reconocimiento de objetos y segmentación de imágenes médicas.

Genética

Clustering jerárquico de ADN para ver relaciones evolutivas.

Algoritmos más utilizados

Tarea	Qué hace	Algoritmos
Clustering	Agrupa datos por similitud	K-means, jerárquico (aglomerativo y divisivo), Modelos de Mezcla Gaussiana
Asociación	Encuentra reglas si-entonces entre elementos	Apriori, Eclat, FP-Growth
Reducción de dimensionalidad	Reduce atributos conservando la información	PCA, SVD, autocodificadores

K-means: conceptos

Clustering exclusivo (duro): cada punto pertenece a un único clúster, el del centroide más cercano. K lo define el usuario.

1 · Inicializar
Elegir K centroides iniciales (idealmente con K-means++).

2 · Asignar
Cada punto va al centroide más cercano.

3 · Actualizar
Cada centroide pasa a ser la media de su grupo.

4 · Repetir
Hasta que las asignaciones ya no cambien.

Fundamentos matemáticos

$$J = \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2$$

Minimizar la suma de distancias al cuadrado de cada punto a su centroide. Con C_k el grupo k y μ_k su media: μ_k = (1/|C_k|) Σ x.

- La función objetivo J no aumenta en cada iteración: converge a un mínimo local.
- El resultado depende de la inicialización: usar K-means++ y varias ejecuciones.
- Costo por iteración aproximado: O(n·K·d).
- Para elegir K: método del codo o coeficiente de silueta.

Aplicaciones, ventajas y desventajas

Usos habituales: segmentación de mercado · agrupación de documentos · segmentación de imágenes · compresión de imágenes (reducir la paleta de colores a K centroides).

Ventajas

- Simple y fácil de implementar
- Rápido y escalable a muchos datos
- Suele converger en pocas iteraciones
- Centroides fáciles de interpretar

Desventajas

- Hay que fijar K de antemano
- Sensible a la inicialización y a los valores atípicos
- Supone clústeres esféricos y de tamaño similar
- Requiere variables numéricas y escaladas

Selección del número óptimo de K

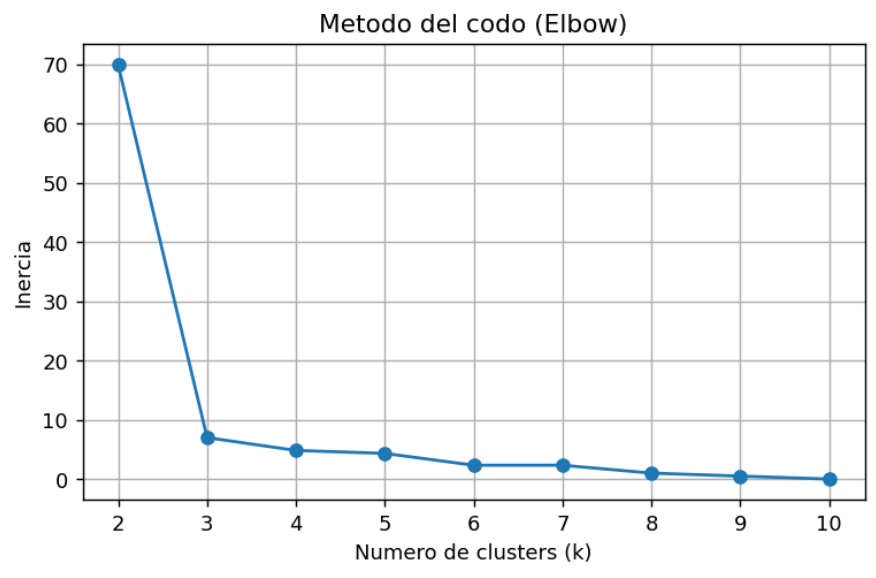
Método del codo (elbow)

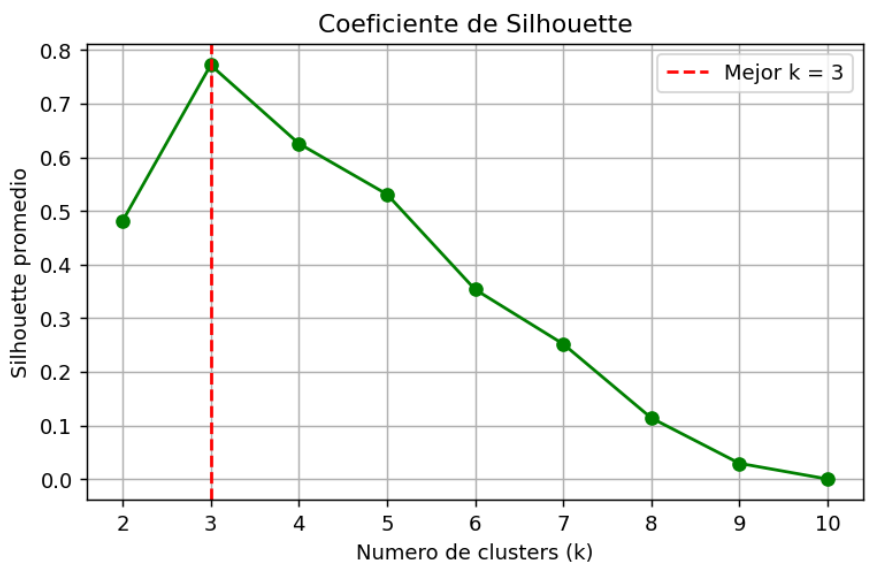
Se calcula J(K) para K = 1…Kmax y se grafica. J siempre disminuye al crecer K; el K óptimo está donde la mejora deja de ser notable (el «codo»). Para automatizarlo se toma el punto con mayor distancia perpendicular a la recta que une el primer y el último punto de la curva.
Ventaja: barato de calcular. Limitación: el codo puede ser ambiguo.

Coeficiente de silueta

Para cada punto: a = distancia media a su propio clúster, b = distancia media al clúster vecino más cercano. s = (b − a) / máx(a, b), con valores entre −1 y 1. Se promedia sobre todos los puntos y se elige el K con mayor silueta media.
Ventaja: criterio explícito y comparable. Limitación: costo $O(n^2)$.

Implementación: elección de K

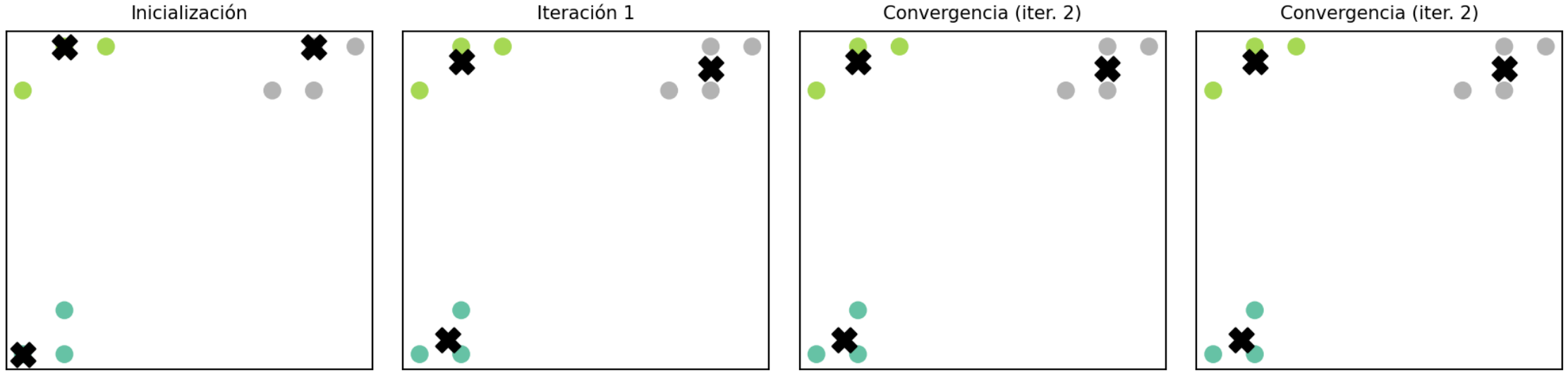




k	Inercia	Silueta
2	69.92	0.4812
3	7.00	0.7721
4	4.83	0.6258
5	4.33	0.5313

- Datos de prueba: nube de 10 puntos en 2D con tres grupos visibles.
- Codo: la inercia cae de 69.92 a 7.00 entre k = 2 y k = 3 y después baja poco.
- Silueta: máxima en k = 3 (0.7721). Ambos métodos coinciden en K = 3.

Implementación: K-means desde cero

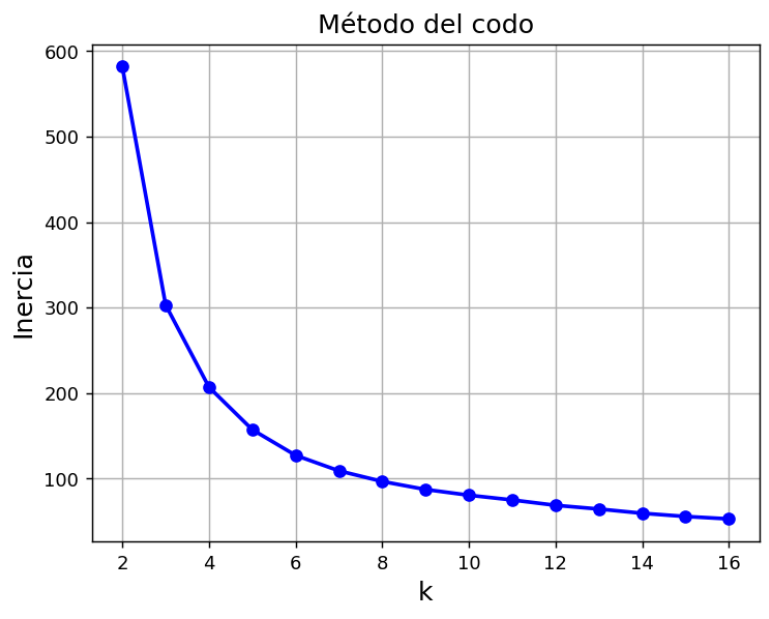


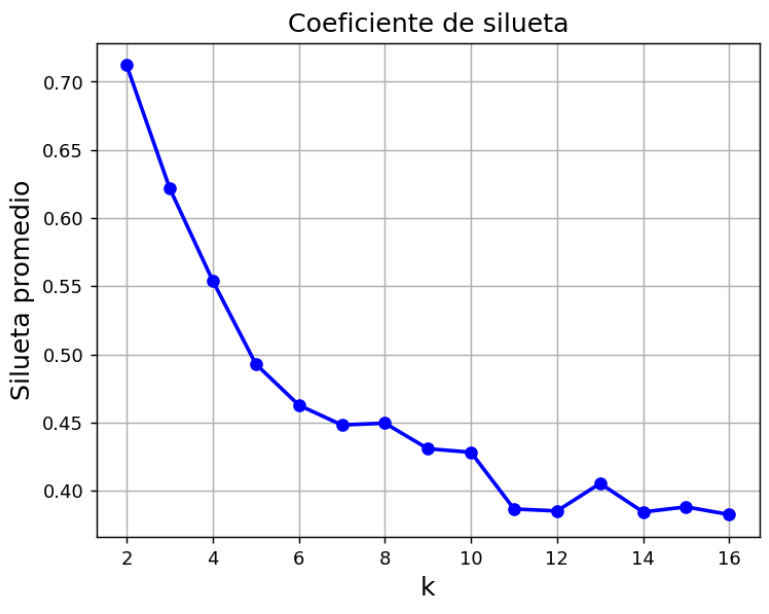
Centroides iniciales: los puntos 0, 3 y 6 de los datos. Cada punto se asigna al centroide más cercano (distancia euclidiana) y cada centroide pasa a ser la media de su grupo. Converge en 2 iteraciones con inercia J = 7.0.

```
for point in datos:                          # asignar
    distances = [np.sqrt(np.sum((point - c)**2)) for c in centroids]
    groups.append(distances.index(min(distances)))
# actualizar: media de los puntos de cada grupo (si queda vacío, conserva su centroide)
if np.allclose(centroids, new_centroids): break   # convergencia
```

Problema 1: reducción de colores

- Cada píxel es un punto (R, G, B) con valores entre 0 y 1: 273,280 píxeles y 96,615 colores distintos.
- K-means se entrena con una muestra de 10,000 píxeles y luego se asigna cada píxel a su centroide.

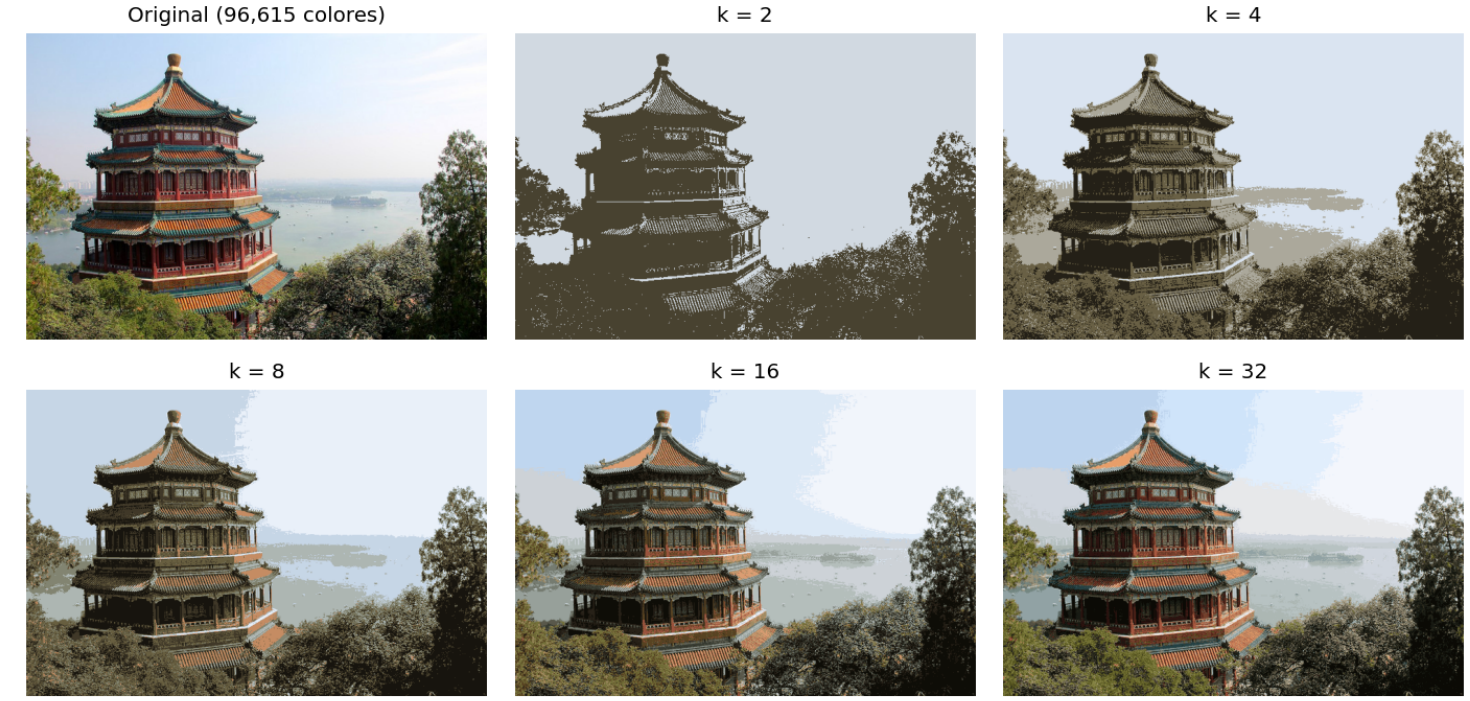




El codo aparece entre k = 4 y 6. La silueta es máxima en k = 2 (cielo claro contra el resto), pero con 2 colores la imagen se pierde. Como los colores de una foto cambian de forma gradual, k se elige como compromiso entre calidad y tamaño.

reducción de colores con k = 2, 4, 8, 16 y 32

Problema 1: resultados



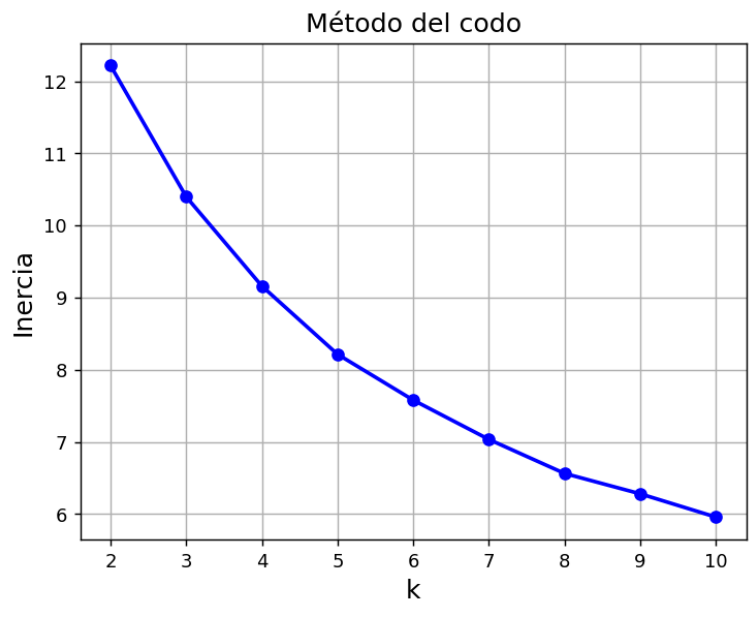
k	Bits/px	KB	% orig.	Error
2	1	33.4	4.2	29.0
4	2	66.7	8.3	17.4
8	3	100.1	12.5	11.2
16	4	133.5	16.7	8.0
32	5	166.9	20.8	6.0

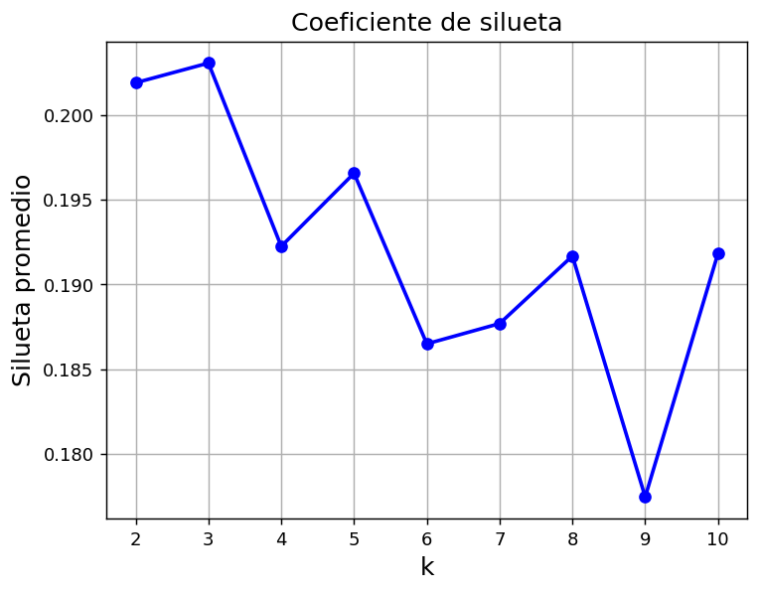
Original: 801 KB (24 bits por píxel). Error: diferencia promedio por canal (0–255).

- Con k = 16 la imagen se ve casi igual y ocupa cerca de 1/6 del espacio.
- Con k = 8 el verde de los árboles se vuelve gris oliva, pero el naranja del techo sobrevive por ser muy distinto al resto.

Problema 2: roles de los Pokémon

- Cada Pokémon es un punto con 6 estadísticas (HP, Attack, Defense, Sp. Atk, Sp. Def, Speed); se usa solo la forma normal de cada especie.
- Se agrupa con el perfil p_ij = x_ij / (suma de sus 6 stats): la fracción del total en cada stat. Así K-means separa roles y no solo débiles de fuertes.
- Los tipos no se usan para agrupar; solo sirven para validar.

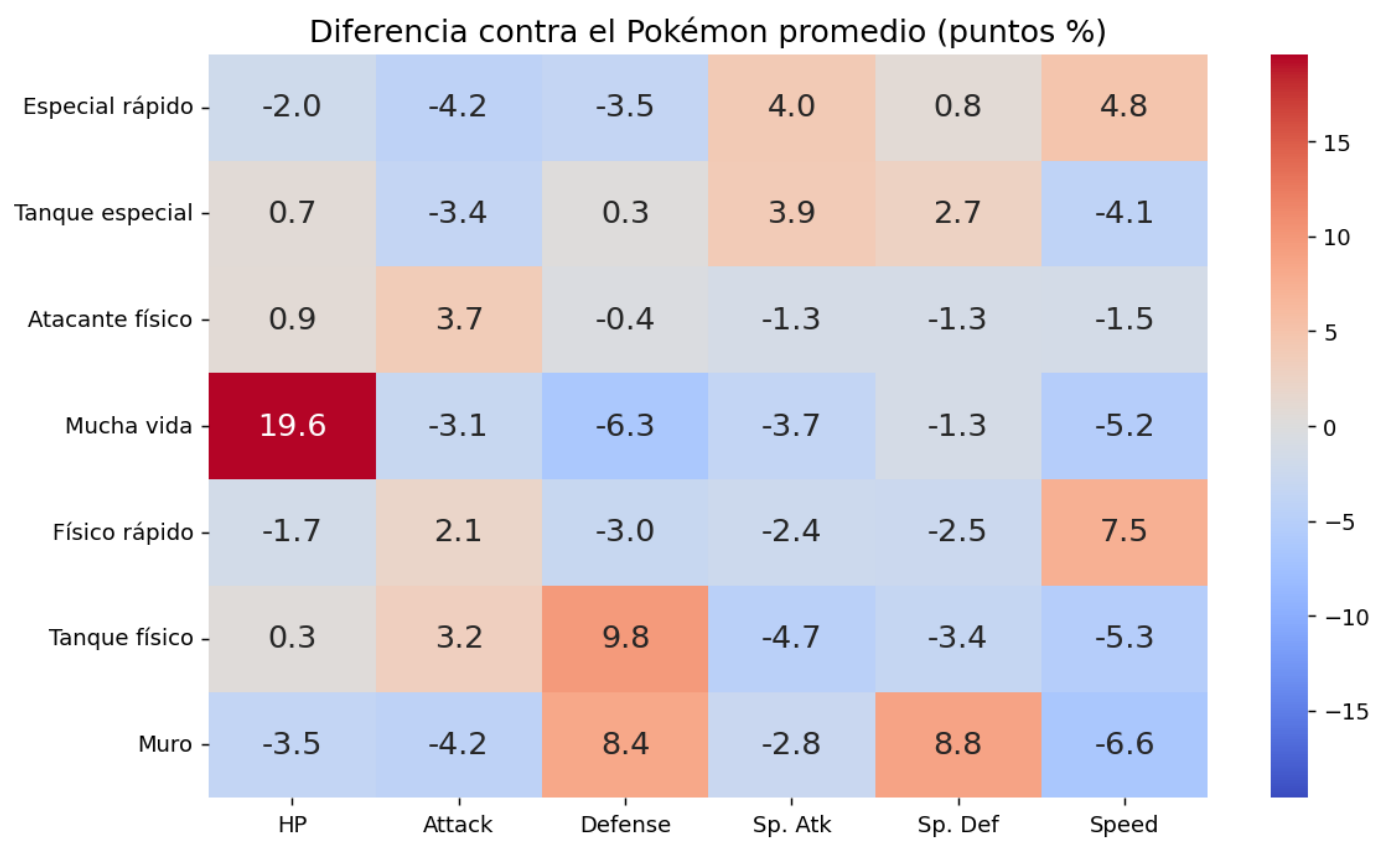




El codo es suave y la silueta casi no cambia (0.18 a 0.20), sin un k claro. Se elige k = 7: con k = 8 aparece un grupo extra de Pokémon equilibrados más que un rol nuevo. La silueta baja (~0.2) refleja el traslape entre roles.

perfil de cada centroide contra el Pokémon promedio

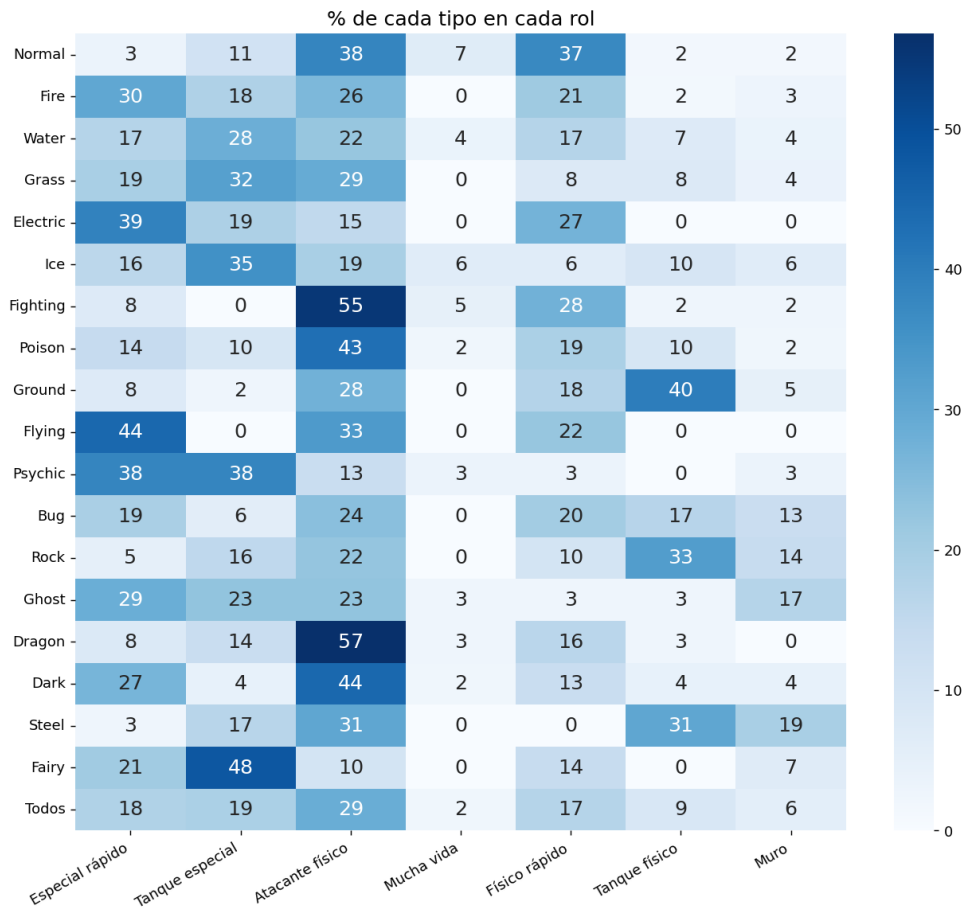
Problema 2: roles encontrados



Rol	N	Ejemplos
Atacante físico	295	Machamp
Tanque especial	195	Togekiss, Blastoise
Especial rápido	185	Alakazam, Gengar
Físico rápido	177	Pikachu, Weavile
Tanque físico	93	Golem, Steelix
Muro	57	Shuckle, Registeel
Mucha vida	23	Snorlax, Blissey

Una regla simple (la stat más destacada) mezcla Pokémon que juegan distinto; K-means compara el perfil completo y encuentra roles que combinan dos stats.

Problema 2: validación con los tipos



- Lucha y Dragón caen en Atacante físico (55–57% contra 29% del total).
- Eléctrico y Psíquico, en Especial rápido (38–39% contra 18%).
- Hada, en Tanque especial (48% contra 19%).
- Tierra, Roca y Acero, en Tanque físico (31–40% contra 9%).
- Acero y Fantasma, en Muro (17–19% contra 6%).

Los roles coinciden con lo que los jugadores esperan de cada tipo. Volador tiene solo 9 Pokémon, así que su fila es poco confiable.

Pruebas y bugs conocidos

- Prueba 1: K-means desde cero sobre 10 puntos, k = 3, converge en 2 iteraciones con J = 7.0.
- Prueba 2: codo y silueta implementados sin librería coinciden en K = 3 (silueta 0.7721).
- Prueba 3: con scikit-learn, la imagen con k = 16 colores baja el error a 8.0 sobre 255; los roles de Pokémon coinciden con sus tipos.

Bugs / limitaciones conocidas

- La versión desde cero usa centroides iniciales al azar y una sola ejecución: el resultado depende de la semilla (no usa K-means++).
- Si un clúster queda vacío, conserva su centroide anterior; la silueta es $O(n^2)$ y en la imagen se calcula con 3,000 píxeles.
- Con silueta baja (~0.2) los roles de Pokémon se traslapan y sus nombres los asigna una persona; solo se usa la forma normal de cada especie.
- En la imagen, las zonas pequeñas con un color único pueden desaparecer. El cuaderno de Pokémon lee datos/pokemon.csv (ruta relativa).

Conclusiones

- K-means resume los datos en k grupos representados por sus centroides. En la nube de 10 puntos, el codo y la silueta programados sin librería coincidieron en k = 3 (J = 7.0).
- Elegir k no se resuelve solo con las métricas: en la imagen la silueta marcó k = 2 y se eligió k = 16 por calidad contra tamaño (1/6 del espacio, error de 8.0 sobre 255).
- Preparar los datos cambia el resultado: con el perfil de cada Pokémon (fracción de su total) K-means encontró 7 roles que coinciden con los tipos, sin haberlos visto.
- Limitaciones: el resultado depende de k y de la inicialización, la silueta es baja (~0.2) en Pokémon porque los roles se traslapan y los nombres de los roles los pone una persona.

Martínez, Esteban

Mi aportación fue la reducción de colores. Aprendí que k no se elige solo con la métrica: la silueta marcó k = 2, pero la imagen se perdía, y con k = 16 quedó casi igual usando 1/6 del espacio. Me sorprendió que el naranja del techo sobreviviera aunque lo usen pocos píxeles.

Osorio, Javier

Logré entender cada paso de K-means (asignar, actualizar y converger) al programarlo sin librería. Mi aportación fue el cuaderno de K-means desde cero y la comparación entre codo y silueta. Lo más difícil fue la silueta, porque compara todos los puntos contra todos y hay que cuidar el caso de un punto solo en su grupo.

Sosa, César

Mi aportación fue el análisis de roles de los Pokémon. Aprendí que preparar los datos importa tanto como el algoritmo: con las stats tal cual K-means solo separaba débiles de fuertes, y con el perfil encontró 7 roles. Validar con los tipos me mostró cómo comprobar grupos sin etiquetas.

formato APA, orden alfabético

Referencias

- Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. En Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (pp. 1027–1035). SIAM.
- Géron, A. (2025). Hands-on machine learning with Scikit-Learn and PyTorch. O'Reilly.
- Greski, L. (s. f.). pokemonData [Conjunto de datos]. GitHub. https://github.com/lgreski/pokemonData
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, 53–65.